

NAGALAND UNIVERSITY

(A Central University Established by an Act of Parliament 1989)

SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

Kohima Campus, Nagaland – 797004



Assignment 3

ITB301: Object Oriented Programming

Submitted By:

Soushnabh Bagchi

Roll / ID: 2505264

Semester: 3rd Sem

Submitted To:

Er Sudipto Patowary

Assistant Professor

Dept. of IT

Date: September 25, 2026

1. Create a class Polynomial that stores the coefficients of a polynomial using a dynamic array.

Code:

```
1  #include <algorithm>
2  #include <cstdint>
3  #include <iostream>
4  #include <string>
5  #include <vector>
6
7  using namespace std;
8
9  struct monomial {
10     int coeff;
11     int power;
12 };
13
14 class Polynomial {
15     vector<monomial> expr;
16     size_t len;
17
18 public:
19     Polynomial(size_t l) { len = l; }
20     ~Polynomial() { cout << "Program executed." << endl; }
21     void insert(int args[2]) { expr.push_back(monomial{args[0],
22     args[1]}); }
23     void derivative() {
24         for (int i = 0; i < len; i++) {
25             if (expr[i].power != 0) {
26                 expr[i].coeff *= expr[i].power;
27                 expr[i].power -= 1;
28             } else {
29                 expr.erase(expr.begin() + i);
30                 len -= 1;
31             }
32         }
33     }
34     void sort() {
35         std::sort(
36             expr.begin(), expr.end(),
37             [](const monomial &a, const monomial &b) { return a.power >
38             b.power; });
39     }
```

```

38  string format() {
39      string ans;
40      for (int i = 0; i < len; i++) {
41          if (expr[i].coeff != 0) {
42              if (expr[i].coeff < 0) {
43                  ans += " - ";
44              } else if (i > 0) {
45                  ans += " + ";
46              }
47              if (abs(expr[i].coeff) != 1) {
48                  ans += to_string(abs(expr[i].coeff));
49              }
50              if (expr[i].power != 0) {
51                  ans += 'x';
52                  if (expr[i].power != 1) {
53                      ans += '^';
54                      ans += to_string(expr[i].power);
55                  }
56              }
57          }
58      }
59      if (ans.size() == 0) {
60          return "0";
61      }
62      return ans;
63  }
64 };
65
66 int main() {
67     int args[2], n;
68     cout << "Enter the number of polynomial terms: ";
69     cin >> n;
70     Polynomial p(n);
71     cout << "Enter the coefficient and polynomial for each term (ax^b):"
72     << endl;
73     for (int i = 0; i < n; i++) {
74         cin >> args[0] >> args[1];
75         p.insert(args);
76     }
77     p.sort();
78     cout << "Original: " << p.format() << endl;
79     p.derivative();

```

```

79  cout << "Derivative: " << p.format() << endl;
80  return 0;
81  }

```

Output:

```

1  Enter the number of polynomial terms: 6
2  Enter the coefficient and polynomial for each term (ax^b):
3  -13 0
4  9 4
5  -2 2
6  6 7
7  8 1
8  -4 5
9  Original: 6x^7 - 4x^5 + 9x^4 - 2x^2 + 8x - 13
10 Derivative: 42x^6 - 20x^4 + 36x^3 - 4x + 8
11 Program executed.

1  Enter the number of polynomial terms: 8
2  Enter the coefficient and polynomial for each term (ax^b):
3  -1 0
4  4 1
5  -5 9
6  7 3
7  12 8
8  -10 2
9  3 6
10 -1 4
11 Original: - 5x^9 + 12x^8 + 3x^6 - x^4 + 7x^3 - 10x^2 + 4x -
12 Derivative: - 45x^8 + 96x^7 + 18x^5 - 4x^3 + 21x^2 - 20x + 4
13 Program executed.

```

2. Create a class that represents a function and differentiates it.**Code:**

```

1  #include <algorithm>
2  #include <cmath>
3  #include <cstdint>
4  #include <iostream>
5  #include <string>
6  #include <vector>

```

C++

```

7
8  using namespace std;
9
10 struct monomial {
11     int coeff;
12     int power;
13 };
14
15 class Polynomial {
16     vector<monomial> expr;
17     size_t len;
18
19 public:
20     Polynomial(size_t l, int args[][2]) {
21         len = l;
22         for (int i = 0; i < l; i++) {
23             this->insert(args[i]);
24         }
25     }
26     void insert(int args[2]) { expr.push_back(monomial{args[0],
27         args[1]}); }
28     void derivative() {
29         for (int i = 0; i < len; i++) {
30             if (expr[i].power != 0) {
31                 expr[i].coeff *= expr[i].power;
32                 expr[i].power -= 1;
33             } else {
34                 expr.erase(expr.begin() + i);
35                 len -= 1;
36             }
37         }
38     }
39     void sort() {
40         std::sort(
41             expr.begin(), expr.end(),
42             [](const monomial &a, const monomial &b) { return a.power >
43                 b.power; });
44     }
45     string format() {
46         string ans;
47         for (int i = 0; i < len; i++) {
48             if (expr[i].coeff != 0) {
49                 if (expr[i].coeff < 0) {

```

```

48         ans += " - ";
49     } else if (i > 0) {
50         ans += " + ";
51     }
52     if (abs(expr[i].coeff) != 1) {
53         ans += to_string(abs(expr[i].coeff));
54     }
55     if (expr[i].power != 0) {
56         ans += 'x';
57         if (expr[i].power != 1) {
58             ans += '^';
59             ans += to_string(expr[i].power);
60         }
61     }
62 }
63 }
64 if (ans.size() == 0) {
65     return "0";
66 }
67 return ans;
68 }
69 double calc(double a) {
70     double sum = 0;
71     for (monomial &m : expr) {
72         sum += m.coeff * pow(a, m.power);
73     }
74     return sum;
75 }
76 };
77
78 int main() {
79     int n = 4;
80     int args[][2] = {{4, 3}, {3, 2}, {-5, 1}, {2, 0}};
81     Polynomial p(n, args);
82     p.sort();
83     cout << "f(x): " << p.format() << endl;
84     p.derivative();
85     cout << "f'(x): " << p.format() << endl;
86
87     double a;
88     cout << "Enter the value of a: ";
89     cin >> a;

```

```

90
91     cout << "Value: " << p.calc(a) << endl;
92     return 0;
93 }

```

Output:

```

1 f(x): 4x^3 + 3x^2 - 5x + 2
2 f'(x): 12x^2 + 6x - 5
3 Enter the value of a: 5
4 Value: 325

```

```

1 f(x): 4x^3 + 3x^2 - 5x + 2
2 f'(x): 12x^2 + 6x - 5
3 Enter the value of a: 6.7
4 Value: 573.88

```

```

1 f(x): 4x^3 + 3x^2 - 5x + 2
2 f'(x): 12x^2 + 6x - 5
3 Enter the value of a: 0
4 Value: -5

```

3. Create a class that represents a function and integrates it using Trapezoidal rule.**Code:**

```

1  #include <algorithm>
2  #include <cmath>
3  #include <cstdint>
4  #include <iomanip>
5  #include <iostream>
6  #include <string>
7  #include <vector>
8
9  using namespace std;
10
11 struct monomial {
12     int coeff;
13     int power;
14 };
15
16 class Polynomial {
17 protected:

```

C++

```

18     vector<monomial> expr;
19     size_t len;
20
21 public:
22     Polynomial(size_t l, int args[][2]) {
23         len = l;
24         for (int i = 0; i < l; i++) {
25             this->insert(args[i]);
26         }
27     }
28     void insert(int args[2]) { expr.push_back(monomial{args[0],
29         args[1]}); }
30     void sort() {
31         std::sort(
32             expr.begin(), expr.end(),
33             [](const monomial &a, const monomial &b) { return a.power >
34                 b.power; });
35     }
36     string format() {
37         string ans;
38         for (int i = 0; i < len; i++) {
39             if (expr[i].coeff != 0) {
40                 if (expr[i].coeff < 0) {
41                     ans += " - ";
42                 } else if (i > 0) {
43                     ans += " + ";
44                 }
45                 if (abs(expr[i].coeff) != 1) {
46                     ans += to_string(abs(expr[i].coeff));
47                 }
48                 if (expr[i].power != 0) {
49                     ans += 'x';
50                     if (expr[i].power != 1) {
51                         ans += '^';
52                         ans += to_string(expr[i].power);
53                     }
54                 }
55             }
56             if (ans.size() == 0) {
57                 return "0";
58             }
59         }
60         return ans;

```

```

59     }
60 };
61
62 class Integrate : public Polynomial {
63 public:
64     Integrate(const Polynomial &other, double a, double b, int divs)
65         : Polynomial(other) {
66         this->divs = divs;
67         this->a = a;
68         this->b = b;
69     }
70     int divs;
71     double a, b;
72     double trapezoidal() {
73         double h = (b - a) / divs;
74         double sum = 0;
75         int mul;
76         for (int i = 0; i <= divs; i++) {
77             if (i == 0 || i == divs) {
78                 mul = 1;
79             } else {
80                 mul = 2;
81             }
82             for (int j = 0; j < len; j++) {
83                 sum += mul * expr[j].coeff * pow(i * h, expr[j].power);
84             }
85         }
86         return sum * (h / 2);
87     }
88 };
89
90 int main() {
91     int n = 1;
92     int args[][2] = {{1, 2}};
93     Polynomial p(n, args);
94     p.sort();
95     cout << "f(x): " << p.format() << endl;
96
97     int divs;
98     cout << "Enter the number of divisions: ";
99     cin >> divs;
100    Integrate i(p, 0, 2, divs);

```

```
101
102     cout << "Integration: " << fixed << setprecision(15) <<
103         i.trapezoidal()
104         << endl;
105     return 0;
106 }
```

Output:

```
1 f(x): x^2
2 Enter the number of divisions: 1
3 Integration: 4.000000000000000
```

```
1 f(x): x^2
2 Enter the number of divisions: 999999999
3 2.666666666666666204
```